14. Design High-Availability Across Regions

Key Terms:

Term	Full Name	What it does
RTO	Recovery Time Objective	Max acceptable downtime after failure
RPO	Recovery Point Objective	Max acceptable data loss (time-based)
Active-Active	Multi-region deployment	Both regions serve traffic simultaneously
Active-Passive	Hot standby	Secondary region waits as backup
GRS	Geo-Redundant Storage	Azure storage replicated across regions
CAP	Consistency, Availability, Partition tolerance	Theorem: can only guarantee 2 of 3
WAF	Web Application Firewall	Protects against web exploits

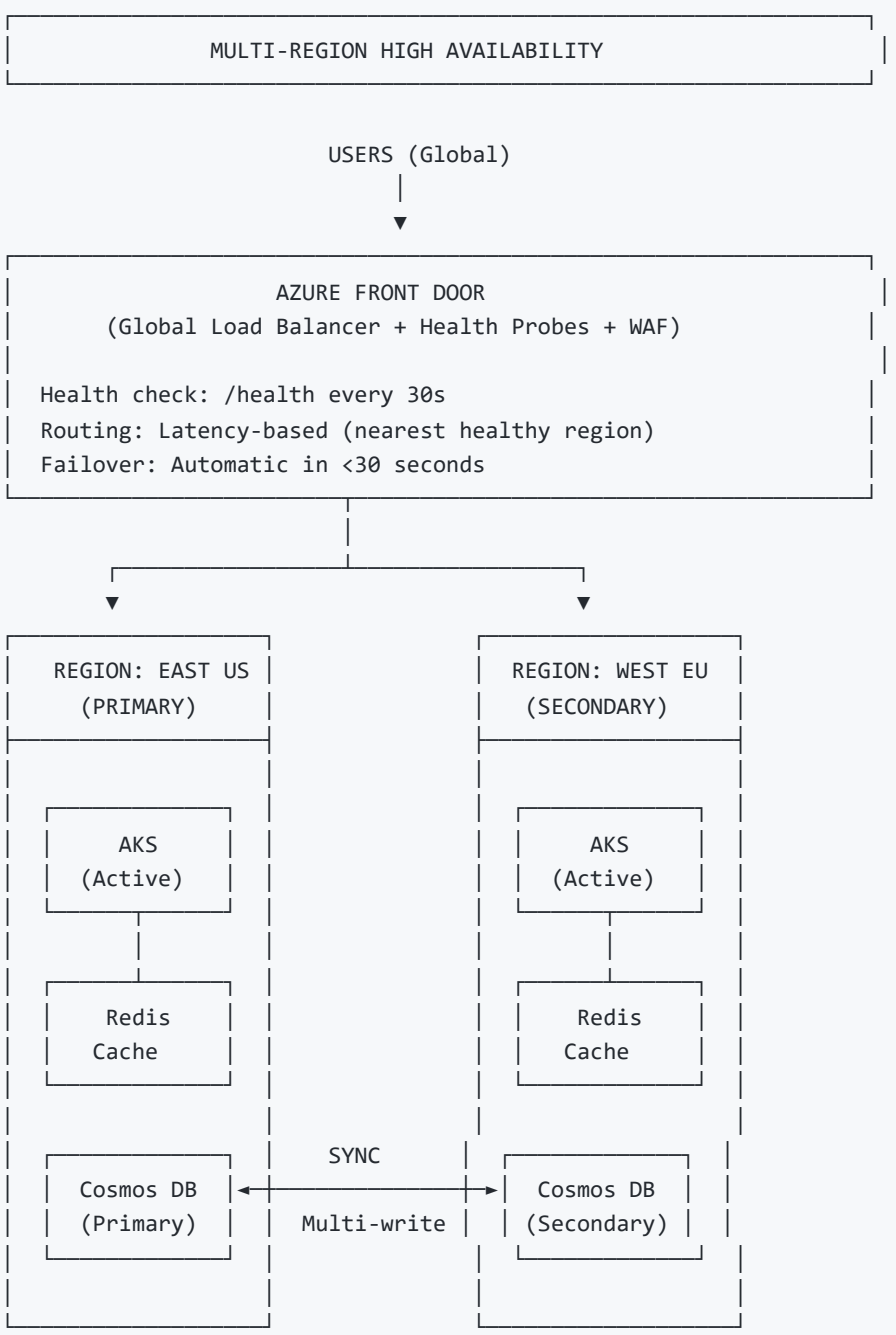
Problem Statement

Design a system that remains available even when an entire Azure region fails, achieving 99.99%+ uptime.

Clarifying Questions to Ask

1. **RTO/RPO**: Recovery time and data loss tolerance?
2. **Traffic pattern**: Global users or region-specific?
3. **Data consistency**: Strong or eventual consistency acceptable?
4. **Cost constraints**: Budget for multi-region?
5. **Compliance**: Data residency requirements?
6. **Failover**: Automatic or manual failover?
7. **Active-active**: Both regions serve traffic or standby?

High-Level Design



API Design

Region Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/regions`	List all regions and their status

`GET`	`/api/v1/regions/{region}/health`	Get region health details
`POST`	`/api/v1/failover`	Trigger manual failover
`GET`	`/api/v1/failover/history`	Get failover event history

Replication APIs

Method	Endpoint	Description
`GET`	`/api/v1/replication/status`	Get replication lag per region
`PUT`	`/api/v1/replication/config`	Update replication policy

Traffic Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/traffic/distribution`	Get traffic routing weights
`PUT`	`/api/v1/traffic/distribution`	Update traffic weights between regions
`PUT`	`/api/v1/traffic/drain/{region}`	Drain traffic from a region

Deployment Patterns

```
ACTIVE-PASSIVE:
┌─────────────────────────────────────────────────────────────┐
│  Primary handles all traffic                                │
│  Secondary is standby (data replicated)                     │
│  Failover: Switch DNS/traffic to secondary                  │
│                                                             │
│  Pros: Simpler, cheaper                                     │
│  Cons: Wasted secondary capacity, slower failover           │
│  Best for: Cost-sensitive, lower availability needs         │
└─────────────────────────────────────────────────────────────┘

ACTIVE-ACTIVE:
┌─────────────────────────────────────────────────────────────┐
│  Both regions serve traffic simultaneously                  │
│  Users routed to nearest region                             │
│  Failover: Automatic, near-instant                          │
│                                                             │
│  Pros: Better latency, instant failover, full utilization   │
│  Cons: Complex, data consistency challenges                 │
│  Best for: Global users, high availability requirements     │
└─────────────────────────────────────────────────────────────┘
```

Data Replication Strategies

```
SYNCHRONOUS REPLICATION:
  Write → Primary → Secondary → Ack to client
  RPO: 0 (no data loss)
  Cons: Higher latency, reduced availability

ASYNCHRONOUS REPLICATION:
  Write → Primary → Ack to client → Secondary (async)
  RPO: Seconds to minutes (some data loss possible)
  Pros: Lower latency, higher availability

COSMOS DB MULTI-REGION:
  - Configurable consistency levels
  - Automatic failover
  - Multi-write for active-active
  - <10ms replication latency
```

Failover Handling

```
AUTOMATIC FAILOVER SEQUENCE:
┌─────────────────────────────────────────────────────────────┐
│  T+0s:   Health probe fails in primary region               │
│  T+30s:  3 consecutive failures, region marked unhealthy    │
│  T+31s:  Traffic automatically routes to secondary          │
│  T+60s:  Alert sent to operations team                      │
│                                                             │
│  Total failover time: ~30-60 seconds                        │
└─────────────────────────────────────────────────────────────┘

HEALTH CHECK ENDPOINT:
  GET /health
  {
    "status": "healthy",
    "checks": {
      "database": "ok",
      "cache": "ok",
      "dependencies": "ok"
    }
  }
```

Key Components

Component	Purpose	Azure Service
Global LB	Route to healthy region	Azure Front Door
DNS failover	Automatic DNS updates	Traffic Manager

Database	Multi-region replication	Cosmos DB
Cache	Regional cache	Azure Cache for Redis
Storage	Geo-redundant storage	Azure Blob (GRS)
Messaging	Cross-region events	Event Grid / Service Bus

Trade-offs

Decision	Option A	Option B	Recommendation
Pattern	Active-Passive	Active-Active	**Active-Active** - better availability
Consistency	Strong	Eventual	**Eventual** - for availability
Failover	Manual	Automatic	**Automatic** - faster recovery
Replication	Sync	Async	**Async** - better latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Load Balancer	Global traffic distribution via Azure Front Door	Tech Guide
Raft Consensus	Leader election and state replication across regions	Tech Guide
Circuit Breaker	Prevent cascade failures during regional outages	Tech Guide

How to Explain It Well

> **Opening**: "High availability requires eliminating single points of failure. I'll design for 99.99% uptime with automatic failover under 60 seconds."

> **Key talking points**:
>
> - "Azure Front Door provides global load balancing with health probes"
> - "Active-active: both regions serve traffic, instant failover"
> - "Cosmos DB multi-write for consistent data across regions"
> - "Accept eventual consistency for availability (CAP theorem)"
> - "Health endpoints check all dependencies, not just the service"
